

WHATSAPP INFRASTRUCTURE GUIDE

בניית WhatsApp Hub עצמאי

מדריך מלא – מאפס ועד API חי שמדבר עם WhatsApp,
כולל אבטחה, חשיפה לאינטרנט ואינטגרציה לכל פלטפורמה

⌚ זמן ביצוע: ~45 דקות · €3.79 לחודש · יכולת בלתי מוגבלת

מחבר

Noam Nissan

תאריך

מאי 2026

github.com/noamnissan/wa-hub-demo

בניית WhatsApp Hub עצמאי – מדריך מלא

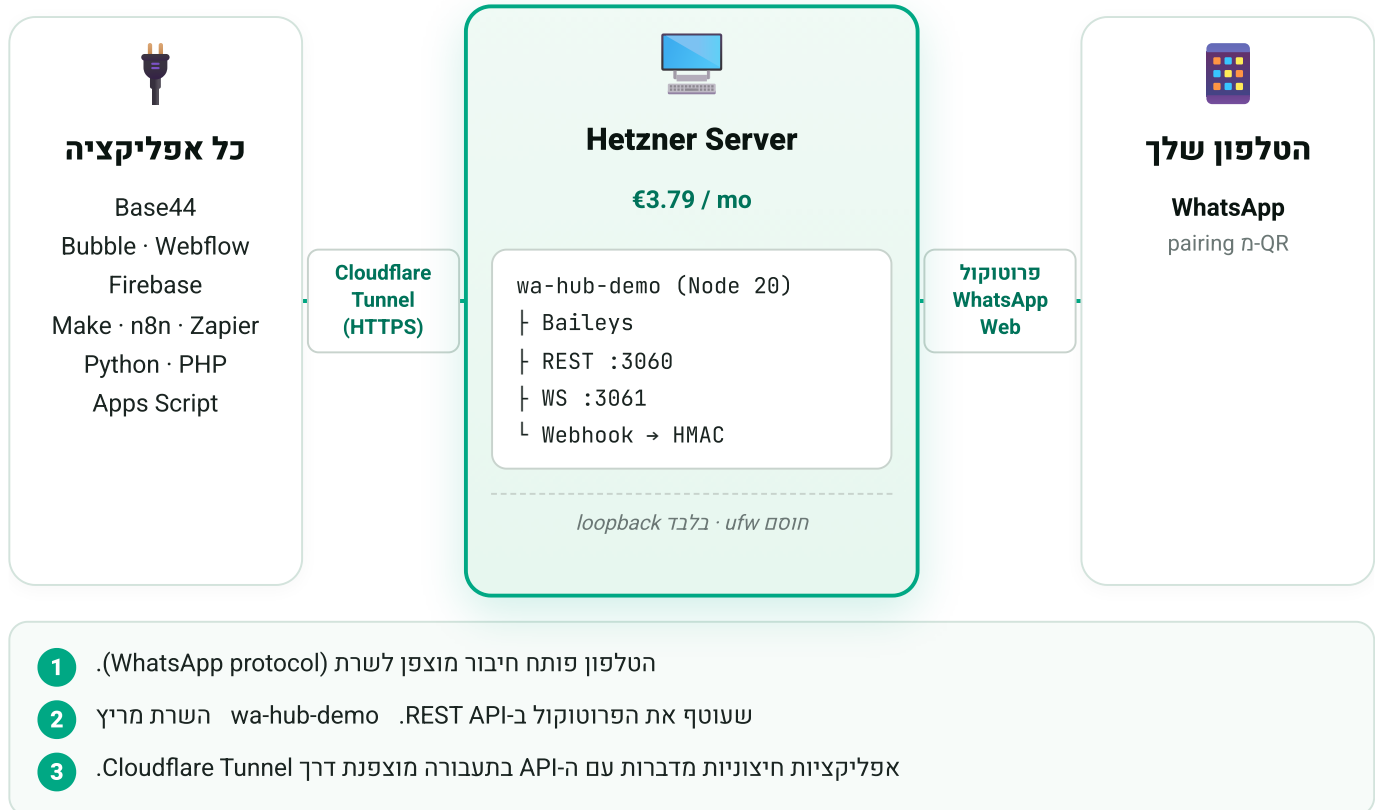
מה תקבל בסוף המדריך הזה: REST API מאובטח שמדבר עם WhatsApp, רץ על שרת שלך, חשוף לאינטרנט בצורה הגיונית, ומדבר עם כל פלטפורמה שאתה רוצה – Base44, Bubble, Firebase, Make, Python, או כל דבר שיוזע HTTP.

זמן ביצוע: ~45 דקות בידיים. **עלות:** €3.79–€3.99 לחודש (Hetzner CAX11 או CX23). **רישיון:** קוד MIT. **מקור:** github.com/Noam13-w/wa-hub-demo

תוכן העניינים

1. [מה אנחנו בונים?](#)
2. [רשימת מה שצריך לפני שמתחילים](#)
3. [שלב 1 – קניית שרת Hetzner](#)
4. [שלב 2 – חיבור SSH ראשון](#)
5. [שלב 3 – בוחרים מסלול](#)
6. [שלב 4 – מסלול A: התקנה ידנית](#)
7. [שלב 4 חלופי – מסלול B: עם Claude Code](#)
8. [שלב 5 – פיירינג WhatsApp](#)
9. [שלב 6 – חשיפה לאינטרנט עם Cloudflare Tunnel](#)
10. [שלב 7 – שימוש מכל מערכת](#)
11. [שלב 8 – אינטגרציה מלאה ל-Base44](#)
12. [שלב 8.5 – שליחה בטוחה: מניעת חסימה](#)
13. [שלב 9 – אבטחה לפרודקשן](#)
14. [טיפים מהשטח](#)
15. [פתרון בעיות נפוצות](#)
16. [שאלות נפוצות](#)
17. [נספח – נהלי חירום לוובר](#)

מה אנחנו בונים?



מה זה כן:

- **HTTP API פשוט** מעל פרוטוקול POST. WhatsApp Web. לשליחה, Webhook לקבלה. כלום מסובך.
- **בעלות מלאה** – הקוד שלך, השרת שלך, ה-token שלך, ההודעות שלך.
- **עלות קבועה** – €3.79 לחודש כל החודש, בלי קשר אם שלחת 10 או מיליון הודעות.
- **קוד פתוח (MIT)** – תפרק, תשנה, תפרסם, תמכור.

■ הקוד שתתקין

הפרויקט `wa-hub-demo` הוא קוד פתוח, זמין ב-github.com/Noam13-w/wa-hub-demo. הוא כולל:

- **שכבת Baileys** שמטפלת בפרוטוקול WhatsApp Web (חיבור, פיירינג, התחברות מחדש אוטומטית, נירמול הודעות)
 - **REST API** ב-Express עם 14 endpoints – שליחת טקסט/תמונה/קובץ/אודיו/מיקום/ריאקציה, פיירינג, סטטוס, ניהול webhook, ניהול קבוצות, בדיקה אם מספר רשום ב-WhatsApp
 - **WebSocket server** שמשדר אירועים בזמן אמת (`message.incoming` , `message.outgoing` , `message.status` עם וי כחול)
 - **Outbound webhooks** עם חתימת HMAC-SHA256 לאימות
 - **Bearer auth** עם `constant-time + rate-limit`
 - **טיפול ב-LID** של 7 Baileys (שדות `senderPn` / `participantPn` / `remoteJidAlt` כדי לחלץ מספר טלפון אמיתי)
 - **systemd unit מוקשח** עם `NoNewPrivileges` , `ProtectSystem=strict` , `CapabilityBoundingSet=` , `MemoryMax=512M` , ריק, `docs/API.md` ו- `docs/ARCHITECTURE.md`
 - **תיעוד API מלא** ב- `docs/API.md` ו- `docs/ARCHITECTURE.md`
- הכל ב-MIT, אתה יכול לעשות איתו מה שתראה.

■ מה זה לא:

- **לא רשמי מ-Meta**. WhatsApp לא מספקים API ישיר כזה. אנחנו מתחזים ל-Linked Device. זה עובד כי הפרוטוקול גלוי, אבל אם תעבירו לקוד שלכם 10K הודעות ביום של ספאם – WhatsApp יחסום את המספר. השתמשו רק להודעות שמישהו ביקש.
- **לא Multi-tenant out-of-the-box**. מספר אחד = שרת אחד = instance אחד. אם רוצים לתת שירות לכמה לקוחות, כל לקוח מקבל instance נפרד (ראו "צעדים הבאים").
- **לא Plug-and-play של Meta WhatsApp Cloud API**. זה משהו אחר – של Meta, דורש אישור עסקי, וכרוך ב-billing per-message. שני העולמות לא חופפים.

■ למי המדריך הזה:

- **מפתחים** שרוצים שליטה מלאה ב-stack של WhatsApp שלהם
- **No-code builders** (Base44, Bubble, Webflow) שרוצים לחבר WhatsApp בלי לקנות תוסף
- **אנשי אוטומציה** (Make, n8n, Zapier) שצריכים אינטגרציה אמينة
- **חברות קטנות** שמעדיפות לשלם €3.79 לחודש קבוע במקום \$0.05 לכל הודעה

רשימת מה שצריך לפני שמתחילים

- ☐ כרטיס אשראי (Hetzner גובה €3.79 בחודש הראשון)
 - ☐ טלפון עם WhatsApp פעיל (להוסיף כ-Linked Device)
 - ☐ חשבון GitHub (חינם, לשכפול הקוד)
 - ☐ חשבון Cloudflare (חינם, אופציונלי – ל-Tunnel)
 - ☐ טרמינל (Mac/Linux: בנוי פנימי. Windows: PowerShell או WSL)
-

שלב 1 – קניית שרת Hetzner

1.1 הרשמה

לכו ל-hetzner.com/cloud ולחצו "Sign Up".

למה Hetzner? היחס מחיר/ביצועים הכי טוב באירופה. €3.79 לחודש = 2 vCPU + 4GB RAM + 40GB SSD + 20TB תעבורה. שווה ערך ל-AWS t4g.medium ב-\$25 לחודש.

1.2 יצירת פרויקט וחיוב

אחרי האימות:

1. הקליקו **"New Project +"** → תנו שם (למשל `wa-hub`)
2. בתפריט שמאל → **"Billing"** → הכניסו פרטי כרטיס
3. בתפריט שמאל → **"Security"** → **"SSH Keys"** → **"Add SSH Key"**

■ 1.3 יצירת מפתח SSH (אם אין לך)

הריצו פקודה אחת בלבד:

על Mac / Linux / WSL:

BASH

```
ssh-keygen -t ed25519 -C "noam-laptop"
```

על Windows PowerShell:

POWERSHELL

```
ssh-keygen -t ed25519 -C "noam-laptop"
```

חשוב: הקלידו את הפקודה **לבד**, בלי להעתיק שורות אחרות אחריה. הפקודה תשאל אתכם 3 שאלות אינטראקטיביות, ואם תדביקו עוד שורה – היא תיכנס בטעות כתשובה לאחת מהן (זה קורה הרבה).

הפקודה תשאל אתכם 3 שאלות. **לכל אחת לחצו Enter** (כל ברירות המחדל בסדר):

1. Enter file in which to save the key → Enter (ברירת מחדל: ~/.ssh/id_ed25519)
2. Enter passphrase (empty for no passphrase) → Enter (בלי סיסמה למפתח)
3. Enter same passphrase again → Enter (אישור)

על passphrase: זה סיסמה שמצפינה את המפתח הפרטי עצמו על הדיסק. עם passphrase תצטרכו להקליד אותה בכל חיבור (אלא אם משתמשים ב-ssh-agent). בלי passphrase נוח יותר. אם המחשב שלכם הוא רק שלכם – Enter פעמיים זה בסדר גמור.

עכשיו **בנפרד** הציגו את המפתח הציבורי:

על Mac / Linux / WSL:

BASH

```
cat ~/.ssh/id_ed25519.pub
```

על Windows PowerShell:

POWERSHELL

```
Get-Content $HOME\.ssh\id_ed25519.pub
```


העתיקו את כל השורה שהודפסה (מתחילה ב- `ssh-ed25519`) → הדביקו ב-Hetzner → תנו שם זיהוי (למשל `laptop`) → שמור.

על C- : זה רק שם שמודבק על המפתח, כמו תווית. רוב המדריכים שמים שם אימייל מהרגל, אבל זה ממש לא חובה ולא מוסיף אבטחה – תן שם שיעזור לך לזהות איזה מפתח זה (`noam-laptop`, `office-mac`, `phone-2026`). המפתח עצמו נוצר מאקראיות קריפטוגרפית של מערכת ההפעלה, לא מהמחרוזת הזאת.

1.4 הזמנת השרת

1. בתפריט שמאל → "Servers" → "Add Server +"

2. **Location:** Falkenstein (גרמניה – הכי קרוב לישראל מבחינת ping, ~60ms). אם אין שם – Nuremberg גם גרמניה ועובד באותה איכות.

3. **Image:** Ubuntu **24.04 LTS** (גם LTS 26.04 עובד, אבל היא חדשה מ-2026/04 ופחות בדוקה. 24.04 מספיקה ל-5 שנים קדימה ונבדקה במדריך הזה לעומק).

4. **Type:** בחרו אחד משני אלה (שניהם עובדים זהה – תלוי במה שזמין באתר באותו רגע):

• **CX23** (x86 / Intel-AMD) – €3.99 לחודש · זמין בכל המיקומים

• **CAX11** (ARM / Ampere) – €3.79 לחודש · זמין רק במיקומים מסוימים (כרגע Nuremberg / Helsinki).

אם הטאב "**Arm64 (Ampere)**" קיים – מצוין, אחרת השתמשו ב-CX23.

5. **Networking:** השאירו ברירת מחדל (IPv4 + IPv6)

6. **SSH keys:** סמנו את המפתח שהוספתם

7. **Volumes / Firewalls / Backups:** דלגו (לא צריך)

8. **Name:** `wa-hub-demo` (או מה שתרצו)

9. לחצו "Create & Buy now"

תוך **30 שניות** השרת יהיה מוכן. שימו לב לכתובת ה-IPv4 – תזדקקו לה.

CX23 vs CAX11 – מה ההבדל בפועל? אפס מבחינת הקוד שלנו. שניהם מריצים Node 20 בלי הבדל.

CAX11 קצת יותר יעיל אנרגטית ו-€0.20 זול יותר. CX23 קצת יותר זמין ובמיקומים יותר. **קחו את מה שזמין**

במיקום הקרוב אליכם – זה לא משנה. הזמינות של ARM משתנה מדי פעם כי Hetzner מנהלים מלאי לפי דרישה.

שלב 2 – חיבור SSH ראשון

זה הצעד שהרבה אנשים נתקעים בו בפעם הראשונה. נעבור לאט. **לא תצטרכו להדביק את המפתח שלכם בשום מקום בשלב הזה** – כבר הדבקנו אותו ב-Hetzner בשלב 1.3, וזה מה שמספיק. ה-SSH client במחשב שלכם ימצא לבד את המפתח הפרטי ב- `~/.ssh/id_ed25519` וישתמש בו.

2.1 – מציאת ה-IP של השרת

בפאנל של Hetzner Cloud → השרת שלכם → תחת "IPv4" תראו כתובת כמו `203.0.113.42`. העתיקו אותה – תזדקקו לה בעוד רגע.

2.2 – פתיחת טרמינל

• **Mac**: Cmd+Space → הקלידו `Terminal` → Enter

• **Linux**: Ctrl+Alt+T (תלוי בהפצה)

• **Windows**: Start → הקלידו `PowerShell` → Enter

◦ Windows 10/11 כולל את `ssh` מובנה. לא צריך להתקין כלום.

2.3 – חיבור

הקלידו (החליפו את `203.0.113.42` ב-IP שלכם):

```
BASH
```

```
ssh root@203.0.113.42
```

בפעם הראשונה תופיע הודעה:

```
The authenticity of host '203.0.113.42 (...)' can't be established.  
ED25519 key fingerprint is SHA256:..  
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

הקלידו `yes` ו-Enter. זאת רק שאלה חד-פעמית – SSH אומר "לא ראיתי את השרת הזה בעבר, אתה בטוח שזה הוא?"

■ 2.4 – אתם בפנים

אם הכל בסדר, תראו:

```
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-XX-generic x86_64)
root@wa-hub-demo:~#
```

הסימן `#` בסוף השורה אומר שאתם מחוברים כ-`root` לשרת. כל מה שתקלידו מכאן ירוץ **על השרת בגרמניה**, לא על המחשב שלכם.

■ 2.5 – בדיקה זריזה שהשרת חי

BASH

```
lsb_release -d
```

BASH

```
free -h
```

BASH

```
curl -fsSL ifconfig.me; echo
```

הראשונה אומרת מה גרסת Ubuntu, השנייה כמה זיכרון, השלישית מאשרת שיש אינטרנט (היא מדפסה את ה-IP שלכם – אותו אחד שהדבקתם למעלה).

■ 2.6 – אם לא הצליח להתחבר

▼ הודעה: Permission denied (publickey)

זה אומר שה-SSH key לא נמצא או לא תקין:

- ודאו שב-Hetzner סימנתם את ה-SSH key בזמן יצירת השרת. אם שכחתם → צריך לאפס.
- ודאו שיצרתם את המפתח עם `ssh-keygen` (שלב 1.3) ושהמפתח קיים: `ls ~/.ssh/id_ed25519`
- (Mac/Linux) או `ls $HOME\.ssh\id_ed25519` (PowerShell). אם המפתח קיים והתקלה ממשיכה – חכו 30 שניות (לפעמים Hetzner צריך זמן להזריק את המפתח לשרת) ותנסו שוב.

▼ שכחתם להוסיף SSH key בזמן יצירת השרת

מהפאנל של Hetzner:

1. כנסו לשרת → "Rescue" → אפסו את סיסמת root
2. תיכנסו עם הסיסמה שקיבלתם: `ssh root@` והקלידו את הסיסמה
3. הדביקו את המפתח הציבורי שלכם לקובץ `~/.ssh/authorized_keys`:

BASH

```
mkdir -p ~/.ssh
echo "ssh-ed25519 AAAA... noam-laptop" >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

(החליפו את `ssh-ed25519 AAAA...` בתוכן האמיתי של `~/.ssh/id_ed25519.pub` שלכם.)

מהפעם הבאה – תוכלו להיכנס עם המפתח, בלי סיסמה.

▼ הודעה: Connection timed out או Connection refused

- ודאו שהשרת באמת רץ בפאנל של Hetzner (אם הוא בסטטוס "starting" – חכו דקה)
- ודאו שלא העתקתם IP לא נכון (IPv6 נראה אחרת – צריך IPv4)
- אם אתם ברשת מוגבלת (אוניברסיטה, חברה) – פורט 22 לפעמים חסום משם

שלב 3 – בוחרים מסלול

מכאן יש **שלוש** דרכים לבנות את ה-Hub:

מסלול A – ידני	מסלול B – Claude Code	מסלול C – Express
קצב	מהיר	מידי (3 דקות)
למידה	מבינים את הזרימה	אפס – קופסה שחורה
התאמה אישית	קלה	קלה (מבקשים מ-Claude)
מומלץ ל-	פעם ראשונה / וובינר	פעם שלישית והלאה
	פרודקשן שאתה סומך עליו	

המלצה לוובינר: התחילו במסלול A (איטי) כדי שהקהל יבין מה קורה. בסוף הראו דמו של C כפיתוי – "מסלול אקספרס בשורה אחת ל-deploy לפרודקשן".

■ מסלול C – Express (אופציה אקספרטית: שורה אחת)

אם יש לכם שרת Hetzner חדש (Ubuntu 24.04 כ-root), והעיקר זה שזה יעבוד מהר:

```
BASH
curl -fsSL https://raw.githubusercontent.com/Noam13-w/wa-hub-demo/main/deploy/install.sh | bash
```

הסקריפט עושה הכל אוטומטית:

- עדכוני מערכת + הקשחת SSH + ufw + fail2ban
 - Node 20 + יצירת npm install + git clone + user `wahub`
 - יצירת סודות אקראיים (`HUB_TOKEN` , `WEBHOOK_SECRET`)
 - התקנת `wa-hub.service` + drop-in לתיקון seccomp של Node 20
 - Cloudflare Tunnel + systemd unit
 - מדפיס לבסוף: **URL ציבורי**, `HUB_TOKEN`, `WEBHOOK_SECRET`, ופקודות מוכנות לפיירינג + שליחה ראשונה
- זמן ביצוע: ~3 דקות. אחרי שהוא רץ, קופצים לשלב 5 (פיירינג).

רוצים לראות מה הסקריפט עושה לפני שאתם רצים? הקובץ נמצא ב- `deploy/install.sh` [ברפרו](#) – 130 שורות קריאות, אפס קסם.

שלב 4 – מסלול A: התקנה ידנית

כל הפקודות מתחת מורצות **על השרת** (אחרי `ssh root@`). במקום עשרה שלבים קטנים, חילקתי לשלושה בלוקים גדולים שכל אחד עומד בפני עצמו.

■ A.1 – הקשחת השרת

עדכונים + נעילת SSH + firewall + fail2ban בריצה אחת. ~3 דקות.

BASH

```
# עדכונים
apt-get update && apt-get -y dist-upgrade && apt-get -y autoremove

# SSH – בלי סיסמאות, בלי סיסמאות – מפתחות בלבד
sed -i 's/^#*PasswordAuthentication */PasswordAuthentication no/' /etc/ssh/sshd_config
sed -i 's/^#*PermitRootLogin */PermitRootLogin prohibit-password/' /etc/ssh/sshd_config
sshd -t && systemctl reload ssh

# Firewall – רק SSH לאינטרנט פתוח
ufw allow 22/tcp comment 'SSH'
ufw default deny incoming && ufw default allow outgoing
yes | ufw enable

# fail2ban – חוסם IPs שמנסים brute-force
apt-get install -y fail2ban
systemctl enable --now fail2ban
```

מה השגנו: שרת שאי אפשר להיכנס אליו בלי המפתח הפרטי שלך, ובוטים שמנסים ננעלים אוטומטית אחרי 3 ניסיונות. ה-Hub עצמו לא חשוף – נטפל בזה דרך Tunnel בשלב 6.

לא להיבהל מה-output: הבלוק הזה מדפיס הרבה מאוד שורות (`apt`) רושם כל חבילה שמתקנת/משדרגת). תוך כדי fail2ban תראו `SyntaxWarning: invalid escape sequence '\s'` – אלה אזהרות קוסמטיות של Python ולא משפיעות על שום דבר. אם בסוף יש לכם `prompt root@wa-hub-demo:~#` – הכל בסדר. אם בנוסף ראיתם `Pending kernel upgrade!` – זה אומר שיש קרנל חדש שיתפוס באתחול הבא, ואין צורך לעשות עם זה כלום עכשיו.

■ A.2 – התקנת Node, יצירת משתמש שירות, ושכפול הקוד

BASH

```
# Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt-get install -y nodejs

# משתמש שירות (בלי --create-home, כי git clone ייצור את התיקייה)
useradd --system --shell /usr/sbin/nologin --home-dir /srv/wa-hub-demo wahub

# שכפול הקוד כ- root (srv/ אינה ניתנת לכתיבה ל-wahub, ואז העברת בעלות
cd /srv
git clone https://github.com/Noam13-w/wa-hub-demo.git
chown -R wahub:wahub /srv/wa-hub-demo

# התקנת הבילות כ-wahub
sudo -u wahub bash -c "cd /srv/wa-hub-demo && npm install --omit=dev"
```

השתמשנו ב- `npm install` כי הוא סלחני יותר עם transitive dependencies (כמו `sharp` שתלוי ב-Baileys). אם תעדיף את ההתנהגות הקפדנית של `npm ci`, ודא שה- `package-lock.json` עדכני: `npm install` פעם אחת מקומית, push, ואז בשרת תוכל להשתמש ב- `npm ci`.

למה לא `--create-home`? הדגל יוצר את `/srv/wa-hub-demo` ריק, וגיט אחר-כך מסרב לעשות clone לתיקייה לא ריקה. הפתרון: יוצרים את המשתמש בלי תיקייה, גיט בונה את התיקייה במהלך ה-clone כ-root, ואז משייכים אותה ל-wahub.

BASH

```
# יצירת סודות אקראיים (256 ביט)
HUB_TOKEN=$(openssl rand -hex 32)
WEBHOOK_SECRET=$(openssl rand -hex 32)

# הצג אותם פעם אחת – שמור במנהל סיסמאות
echo "==== שמרו את הערכים האלה ====="
echo "HUB_TOKEN=$HUB_TOKEN"
echo "WEBHOOK_SECRET=$WEBHOOK_SECRET"
echo "===== "

env. כתיבת #
cat > /srv/wa-hub-demo/.env <<EOF
HUB_NAME=wa-hub
HUB_TOKEN=$HUB_TOKEN
WEBHOOK_SECRET=$WEBHOOK_SECRET
HUB_PORT=3060
WS_PORT=3061
RATE_LIMIT_PER_MIN=120
DATA_DIR=/srv/wa-hub-demo/data
LOG_LEVEL=info
EOF
chown wahub:wahub /srv/wa-hub-demo/.env
chmod 600 /srv/wa-hub-demo/.env

data יצירת תיקיית #
mkdir -p /srv/wa-hub-demo/data
chown -R wahub:wahub /srv/wa-hub-demo/data

# התקנה כ- systemd service, מתאושש אוטומטית
install -m 644 /srv/wa-hub-demo/deploy/wa-hub.service /etc/systemd/system/wa-hub.service
systemctl daemon-reload
systemctl enable --now wa-hub.service

# בדיקה אחרונה – אמור להחזיר {"ok":true, "connection":"qr"}
sleep 4
curl -sS http://127.0.0.1:3060/healthz
```

השירות פעיל! `systemd` יחזיר אותו תוך 5 שניות אם הוא נופל, רץ עם `MemoryMax=512M` כדי לוודא שגם דליפת זיכרון של Baileys לאורך זמן לא תפיל את השרת, ויפעל אוטומטית בכל `boot`.

שלב 4 חלופי – מסלול B: עם Claude Code

אם רוצים לדלג את כל המעלה ולתת ל-AI לעשות:

■ B.1 – התקנת Claude Code על המחשב שלכם

BASH

```
npm install -g @anthropic-ai/claude-code
claude login
```

■ B.2 – תכנסו לשרת שלכם דרך Claude

חשבו על זה כשני טרמינלים שעובדים במקביל: אחד SSH לשרת, השני Claude Code על המחשב שלכם.

על המחשב המקומי:

BASH

```
mkdir wa-hub-prep && cd wa-hub-prep
claude
```

ובתוך Claude הקלידו:

המטרה שלי: על שרת Hetzner חדש (x86 – Ubuntu 24.04 או ARM, IP=<X.X.X.X),
להתקין את הפרויקט <https://github.com/Noam13-w/wa-hub-demo>, להריץ אותו
כ-systemd service תחת משתמש wahun, ולהקים Cloudflare Quick Tunnel
שיחשוף אותו לאינטרנט.

תעבוד עם Bash דרך ssh root@<X.X.X.X>. תאשר איתי כל שלב לפני שאתה מבצע.
תתחיל באודיט מצב השרת.

Claude Code יעבור צעד-צעד, ישאל לאישור לפעולות risky, וידפיס בסוף את ה-token וה-URL הציבורי. בערך 10 דקות.

מאיפה Claude יודע איך? הוא קורא את הקוד והמדריך הזה. הוא לא מנחש – הוא רואה את אותם השלבים שאתם רואים, ומבצע אותם.

שלב 5 – פיירינג WhatsApp

ה-Hub רץ, אבל הוא עוד לא מחובר לאף מספר. עכשיו נחבר.

🕒 **זה החלק הטריקי:** ל-QR יש תוקף של **60 שניות** ואז הוא מתחלף. לכן צריך **להכין הכל מראש** – טלפון פתוח, חלון PowerShell מוכן – ואז בלחיצה אחת על השרת כל השאר רץ במהירות. הסיקוונס המלא: ~10 שניות.

5.1 – להכין הכל מראש (לפני שמייצרים QR!)

(א) **על הטלפון:** פתח WhatsApp → הגדרות → מכשירים מקושרים → קישור מכשיר. המצלמה תפתח ותחכה למשהו לסרוק. **השאר את זה פתוח.**

(ב) **על המחשב המקומי:** פתח חלון PowerShell **חדש** (לא בשרת!). הקלד את הפקודה הבאה אבל **אל תלחץ Enter עדיין** – נריך אותה ברגע הנכון:

POWERSHELL

```
scp root@<IP>:/tmp/qr.png $HOME\qr.png; Start-Process $HOME\qr.png
```

(החלף את — בכתובת השרת שלך. המקבילה ל-Mac/Linux:

```
(. scp root@:/tmp/qr.png ~/qr.png && open ~/qr.png
```

(ג) **בחלון ה-SSH לשרת,** הקלד גם פה אבל **אל תלחץ Enter עדיין:**

BASH

```
curl -sS -H "Authorization: Bearer $TOKEN" http://127.0.0.1:3060/api/instance/qr.png -o /tmp/qr.png
```

(וודא שעדיין יש לך **\$TOKEN** טעון. אם פתחת SSH מחדש, הרץ קודם:

```
( TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)
```

■ 5.2 – הסיקוונס המהיר

עכשיו שהכל מוכן:

1. בחלון ה-Ssh → Enter (curl שומר את ה-QR ב- /tmp/qr.png)

2. בחלון PowerShell → Enter (scp מוריד, ו-Start-Process פותח את התמונה)

3. בטלפון → סרוק את התמונה שנפתחה במחשב

תוך 2-3 שניות תראה בלוג של השרת `WhatsApp connected`, ובטלפון תראה את ההתקן בליסט "מכשירים מקושרים".

אם פספסת ב-60 שניות: פשוט תריץ שוב את אותן 2 פקודות (Enter ב-SSH → Enter ב-PowerShell). ה-Hub מייצר QR חדש כל 60 שניות אוטומטית.

■ 5.3 – בדיקה: שולחים הודעה לסוכן AI שלי

על השרת:

BASH

```
TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)
curl -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"to":"35796699735","text":"ניסן! יצאתי api של נעם"}' \
  http://127.0.0.1:3060/api/messages/send/text
```

המספר 35796699735 הוא הסוכן AI שלי. שלח לו את ההודעה כדי לבדוק שה-API שלך עובד – אל תדאג לספאם, הוא יודע להתמודד וגם יענה לכם בתגובה משעשעת 😊
בהמשך, כשתרצה לשלוח למספרים אחרים (לקוחות, חברים), פשוט החלף את ה-`to` במספר היעד. הפורמט: קוד מדינה בלי + ובלי 0 בהתחלה, ואז המספר. דוגמה – `0585802298` הופך ל-`972585802298`.

■ 5.4 – איפוס אם ניתקתם בטעות (או בכוונה)

אם ניתקתם את המכשיר מהטלפון (WhatsApp → מכשירים מקושרים → לחיצה על ההתקן → ניתוק), ה-Hub יזהה ויפסיק להתחבר מחדש (התנהגות מכוונת – לא רוצים שהוא ינסה לחזור בלי הסכמתכם). תראו בלוג: `loggedOut`

```
ERROR: Device was logged out from the phone. Clear /data/auth and re-pair.
```

לאפס ולקבל QR חדש:

BASH

```
TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)
curl -X POST -H "Authorization: Bearer $TOKEN" http://127.0.0.1:3060/api/instance/logout
```

זה מוחק את `data/auth`, מפעיל מחדש את Baileys, ותוך 3-4 שניות יש QR חדש. אחר-כך חזרו לשלב 5.1 לסריקה מחדש.

למה הם לא מתחברים אוטומטית? במכוון. אם הטלפון הראשי החליט שלא רוצה את ההתקן הזה יותר, רוצים שתאשרו במודע שזה היה תאונה ולא מישהו חיצוני שהשתלט. אותו דבר קורה במקרה של `connectionReplaced` (קוד 440) – שני sessions לא מסתחים זה עם זה אינסוף.

שלב 6 – חשיפה לאינטרנט עם Cloudflare Tunnel

עד עכשיו ה-API רץ רק מקומית (127.0.0.1) / Base44. כל אפליקציה חיצונית לא יכולה להגיע. בואו ננתב את זה החוצה – בלי לפתוח פורט בכלל.

6.1 – התקנת cloudflared

BASH

```
ARCH=$(dpkg --print-architecture) # מחזיר אוטומטית: amd64 ל-CX23, arm64 ל-CAX11
curl -fsSL -o cloudflared.deb \
  "https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-
  linux-$ARCH.deb"
dpkg -i cloudflared.deb
cloudflared --version
```

6.2 – Quick Tunnel (URL רנדומלי, ללא חשבון)

הכי קל לדמו:

BASH

```
cloudflared tunnel --url http://127.0.0.1:3060
```

תוך 5 שניות תראו:

```
Your quick Tunnel has been created! Visit it at (it may take some time to be reachable):
https://soft-clouds-rapidly-eat.trycloudflare.com
```

זה ה-URL הציבורי שלכם. בדקו ממחשב אחר:

BASH

```
curl https://soft-clouds-rapidly-eat.trycloudflare.com/healthz
```

חיסרון Quick Tunnel: ה-URL רנדומלי ומשתנה בכל restart. אם השרת נופל ועולה – צריך לעדכן את ה-URL בכל מקום שמשתמש בו. **מצוין לדמו**, פחות לפרודקשן.

■ 6.3 Quick Tunnel כ- systemd service (לא ייפול)

כדי שה-Tunnel יישאר חי גם אחרי שהטרמינל נסגר:

BASH

```
cat > /etc/systemd/system/cloudflared-wahub.service <<'EOF'
[Unit]
Description=Cloudflare Quick Tunnel for wa-hub
After=network-online.target wa-hub.service
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/bin/cloudflared tunnel --no-autoupdate --url http://127.0.0.1:3060
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal
DynamicUser=yes

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now cloudflared-wahub.service

# המתינו 6 שניות, אז שלפו את ה-URL מהלוג:
sleep 6
journalctl -u cloudflared-wahub.service -n 30 --no-pager | \
  grep -Eo 'https://[a-z0-9-]+\.\trycloudflare\.com' | head -1
```

■ 6.4 Named Tunnel (URL קבוע על דומיין שלך)

ל-פרודקשן, אתה רוצה URL יציב כמו `https://api.yourdomain.com` במקום `hughes-random-words.trycloudflare.com` שמתחלף בכל `restart`.

מה צריך לפני שמתחילים

- **דומיין** (מ-Cloudflare Registrar, GoDaddy, Namecheap, או כל רושם אחר). אפילו דומיין זול ב-\$10/שנה עובד.
- **הדומיין מנוהל ב-Cloudflare DNS**. אם רכשת ב-Cloudflare – אוטומטית. אם במקום אחר – צריך להוסיף את הדומיין ל-Cloudflare ולעדכן את ה-`nameservers`. הדרכה: developers.cloudflare.com/dns/zone-setups/full-setup.

מה לבחור: `subdomain` או `root`?

מומלץ: `subdomain` ייעודי, למשל `api.yourdomain.com` או `wa.yourdomain.com`.

) root (<code>example.com</code>		) subdomain (<code>api.example.com</code>	
✗		✓	בידוד מהאתר הראשי
✗ (דורש redirect)		✓	יכול לחיות ביחד עם אתר wordpress / וכו' על השרת הראשי
רק לפרויקטים API-only		✓	נהוג בתעשייה

בדוגמאות מתחת אני אשתמש ב- `api.example.com`. תחליפו לדומיין שלכם.

שלב-אחר-שלב

1. התחברות ראשונית של cloudflared לחשבון שלך:

BASH

```
cloudflared tunnel login
```

יוצא URL לדפדפן. פתחו אותו במחשב הראשי שלכם, התחברו ל-Cloudflare, ובחרו את הדומיין שלכם מהרשימה. אחרי האישור – `~/cloudflared/cert.pem` נכתב על השרת.

2. יצירת tunnel בשם:

BASH

```
cloudflared tunnel create wa-hub
```

הפלט יראה משהו כמו:

```
Created tunnel wa-hub with id 4f7c9d3e-abcd-1234-5678-90abcdef1234
```

שמרו את ה-ID tunnel הזה. קוראים לו – במשך כל ה-instructions.

3. ניתוב DNS – מחבר את הסאב-דומיין ל-tunnel:

BASH

```
cloudflared tunnel route dns wa-hub api.example.com
```

(החליפו `api.example.com` בסאב-דומיין שלכם.) הפקודה הזאת יוצרת אוטומטית רשומת CNAME ב-Cloudflare שמצביעה לתעבורה לתוך ה-tunnel – אתם לא צריכים לפתוח את Cloudflare ולעשות זה ידנית.

אם תרצו עוד סאב-דומיינים על אותו tunnel (למשל גם `wa.example.com` או `api.example.co.il`), פשוט הריצו את הפקודה הזאת שוב לכל אחד.

4. קונפיגורציה של ה-tunnel – איזה traffic איפה ינתב:

BASH

```
mkdir -p /etc/cloudflared
```

BASH

```
nano /etc/cloudflared/config.yml
```

הדביקו (החליפו – ב-ID tunnel שלכם, ו- `api.example.com` בסאב-דומיין):

YAML

```
tunnel: <TID>
credentials-file: /root/.cloudflared/<TID>.json

ingress:
  - hostname: api.example.com
    service: http://127.0.0.1:3060
  - service: http_status:404
```

הסבר על הבלוק `ingress`:

- כל בקשה לכתובת `api.example.com` עוברת ל-Hub שלכם ב- `127.0.0.1:3060` ✓
 - כל בקשה אחרת מקבלת 404 (fallback ברירת מחדל – חובה!)
- שמרו (`Ctrl+X` , `Enter` , `Ctrl+0`) ב-nano).

5. אם יש לכם כבר Quick Tunnel רץ כ-systemd – עצרו אותו:

BASH

```
systemctl disable --now cloudflared-wahub.service 2>/dev/null
```

(אחרת תהיו עם 2 tunnels שמסתחבים על אותו פורט).

6. התקנה כ-systemd service של cloudflared הרשמית:

BASH

```
cloudflared service install
```

ה-CLI מתקין את עצמו כ-systemd עם השם `cloudflared.service`. הוא יקרא אוטומטית את `/etc/cloudflared/config.yml`.

7. הפעלה ובדיקה:

BASH

```
systemctl enable --now cloudflared
```

BASH

```
sleep 5 && systemctl status cloudflared --no-pager
```

צריך לראות `active (running)`. לוג שגיאות:

BASH

```
journalctl -u cloudflared -n 50 --no-pager
```

8. בדיקה ממחשב חיצוני:

BASH

```
curl https://api.example.com/healthz
```

צריך להחזיר `{"ok":true, ...}`. אם כן – **הצלחתם!** ה-Hub שלכם זמין עכשיו ב-
באופן קבוע, מאחורי Cloudflare (DDoS protection ו-SSL – הכל אוטומטי, `https://api.example.com`
(חינם).

עדכוני הסביבה אחרי המעבר ל-Named Tunnel

- עדכנו את ה-WEBHOOK_URL ב-env כדי לא ללכת לאיבוד בכל restart:

BASH

```
sed -i 's|^WEBHOOK_URL=.*|WEBHOOK_URL=https://api.example.com/your-webhook-endpoint|'  
/srv/wa-hub-demo/.env  
systemctl restart wa-hub
```

- בכל מקום שהשתמשתם ב-URL הזמני (Base44 secrets, Bubble API config וכו') – תחליפו ל-
`https://api.example.com`. זה לא יותר ישתנה.

אם אתם רוצים יותר מ-tunnel אחד

מבנה ה-config.yml מאפשר routing מתוחכם – כל hostname לפורט אחר, או אפילו לשרת אחר. דוגמה:

YAML

```
ingress:  
- hostname: api.example.com  
  service: http://127.0.0.1:3060  
- hostname: admin.example.com  
  service: http://127.0.0.1:8080  
- hostname: db.example.com  
  service: tcp://127.0.0.1:5432  
- service: http_status:404
```

זה אומר: tunnel אחד יכול לתת שירות לכמה אפליקציות במקביל, כל אחת על subdomain שלה.

שלב 7 – שימוש מכל מערכת

ה-API שלך הוא **REST + JSON + Bearer auth** – הסטנדרט הכי משעמם בעולם. וזה היתרון. כל פלטפורמה שיודעת לעשות HTTP request מסוגלת לדבר איתו.

בכל הדוגמאות:

- ה-URL הציבורי של ה-Tunnel שלך = `HUB_URL`
- ה-Bearer מ-`.env` = `HUB_TOKEN`
- שמור אותם **רק בצד שרת / סודות**, לעולם לא ב-JS של דפדפן

■ 7.1 – Bubble

ב-Bubble Plugins → **API Connector**:

1. **Add another API** → תן שם `WhatsApp Hub`

2. **Authentication**: `Private key in header`

3. **Key name**: `Authorization` · **Key value**: `Bearer`

4. **Add call**

◦ **Name**: `Send Text`

◦ **Method**: `POST`

◦ **URL**: `/api/messages/send/text`

◦ **Body type**: `JSON`

◦ **Body**: `{"to":"","text":""}`

◦ סמן את `to` ו-`text` כ-parameters

ב-workflow: `When Button "Send" is clicked → API call WhatsApp Hub Send Text`

JAVASCRIPT

```
// functions/index.js
const { onRequest } = require("firebase-functions/v2/https");
const { defineSecret } = require("firebase-functions/params");

const HUB_TOKEN = defineSecret("HUB_TOKEN");
const HUB_URL = "https://your-tunnel.trycloudflare.com";

exports.sendWa = onRequest({ secrets: [HUB_TOKEN] }, async (req, res) => {
  const r = await fetch(`${HUB_URL}/api/messages/send/text`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${HUB_TOKEN.value()}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify(req.body),
  });
  res.status(r.status).json(await r.json());
});
```

הזריקו את הסוד: `firebase functions:secrets:set HUB_TOKEN`

Make / n8n / Zapier – 7.3 ■

שליחה:

1. הוסיפו (n8n) **HTTP Node** / (Make) **HTTP Request** / **Webhooks by Zapier**
2. **URL**: `/api/messages/send/text`
3. **Method**: **POST**
4. **Headers**: `Content-Type: application/json` + `Authorization: Bearer`
5. **Body**: `{"to":"+972...", "text":"hi"}`

קבלת webhook:

1. צרו Webhook ב-Make/n8n/Zapier → תקבלו URL
2. רשמו אותו ב-Hub:

BASH

```
curl -X PUT -H "Authorization: Bearer $HUB_TOKEN" \
-H "Content-Type: application/json" \
-d '{"url":"'<your make/n8n webhook url>'", "events":["message.incoming"]}' \
$HUB_URL/api/instance/webhook
```

כל הודעה נכנסת מופיעה בתוך n8n/Make כטריגר אוטומטי.

Node.js / Next.js – 7.4 ■

JAVASCRIPT

```
// app/api/send-wa/route.js (Next.js App Router)
export async function POST(req) {
  const { to, text } = await req.json();
  const r = await fetch(`${process.env.HUB_URL}/api/messages/send/text`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${process.env.HUB_TOKEN}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ to, text }),
  });
  return Response.json(await r.json(), { status: r.status });
}
```

PYTHON

```
import os, requests

HUB_URL = os.environ["HUB_URL"]
HUB_TOKEN = os.environ["HUB_TOKEN"]

def send_whatsapp(to: str, text: str):
    r = requests.post(
        f"{HUB_URL}/api/messages/send/text",
        headers={"Authorization": f"Bearer {HUB_TOKEN}"},
        json={"to": to, "text": text},
        timeout=15,
    )
    r.raise_for_status()
    return r.json()
```

:webhook (FastAPI) קבלת

PYTHON

```
import hmac, hashlib, os
from fastapi import FastAPI, Request, HTTPException

app = FastAPI()
SECRET = os.environ["HUB_WEBHOOK_SECRET"]

@app.post("/wa/incoming")
async def incoming(req: Request):
    body = await req.body()
    got = req.headers.get("x-hub-signature", "")
    want = "sha256=" + hmac.new(SECRET.encode(), body, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(got, want):
        raise HTTPException(401)
    event = await req.json()
    # ... your logic ...
    return {"ok": True}
```

PHP

```
use Illuminate\Support\Facades\Http;

$r = Http::withToken(env('HUB_TOKEN'))
    ->post(env('HUB_URL') . '/api/messages/send/text', [
        'to' => $to,
        'text' => $text,
    ]);
return response()->json($r->json(), $r->status());
```

Google Sheets / Apps Script — 7.7 ■

JAVASCRIPT

```
function sendWhatsApp(to, text) {
    const HUB_URL = PropertiesService.getScriptProperties().getProperty('HUB_URL');
    const HUB_TOKEN = PropertiesService.getScriptProperties().getProperty('HUB_TOKEN');
    return UrlFetchApp.fetch(`${HUB_URL}/api/messages/send/text`, {
        method: 'post',
        contentType: 'application/json',
        headers: { Authorization: `Bearer ${HUB_TOKEN}` },
        payload: JSON.stringify({ to, text }),
    }).getContentText();
}

:sheet-ב שורה לכל הודעה //
function bulkSend() {
    const rows = SpreadsheetApp.getActiveSheet().getDataRange().getValues();
    rows.slice(1).forEach(([phone, message]) => sendWhatsApp(phone, message));
}
```

BASH

```
# /etc/cron.daily/wa-summary.sh
TOKEN=$(cat /etc/hub.token)
curl -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"to":"+972501234567","text":"Daily summary ready"}' \
  https://your-tunnel.trycloudflare.com/api/messages/send/text
```

התובנה: כמעט כל פלטפורמת no-code/low-code היום יודעת לעשות HTTP. ה-Hub שלך הופך לכל אחת מהן ל"ספק WhatsApp" באופן מיידי, בלי לקנות עוד תוסף או תשלום per-message.

שלב 8 – אינטגרציה מלאה ל-Base44 (דרך הצ'אט/Builder)

Base44 מקבל פרק מורחב כי הוא נפוץ מאוד בקהילה של no-code/low-code. במקום לכתוב קוד ידנית, נשתמש ב-Builder של Base44 – הצ'אט-AI שמייצר לכם את הקוד. תכתבו לו prompt אחד טוב, והוא יבנה את ה-entities, ה-functions, וה-UI.

אופציה מתקדמת: אם אתם מפתחים שמעדיפים CLI – ראו 8.7 בסוף הפרק.

■ 8.1 – יצירת פרויקט חדש ב-Base44

1. כנסו ל-app.base44.com (תירשמו אם אין לכם חשבון – חינם)
2. במסך הראשי לחצו "Create new app" (או "פרויקט חדש")
3. תיפתח חלון צ'אט. תקבלו prompt התחלתי – תוכלו לכתוב שם בעברית או באנגלית.

■ 8.2 – ה-Prompt: התקנת הקונקטור ל-WhatsApp Hub

הרעיון: הדביקו את ה-prompt הזה ב-Base44 builder כדי **להתקין את החיבור** ל-Hub – בלי לבנות אפליקציה ספציפית. אחרי שזה רץ, יש לכם 3 פונקציות מוכנות שאפשר לקרוא מכל מקום באפליקציה שלכם. בשלב הבא תגידו ל-Base44 מה לעשות עם זה: "תוסיף כפתור 'שלח WhatsApp' לכל לקוח", "תפתח דף צ'אט", "תגרום לבוט לענות אוטומטית" וכו'.

חשוב – סודות: ה-prompt למטה כולל פלייסהולדרים כמו `{secrets}` בתוך בלוק ה-`secrets`. **אל תערכו את הפלייסהולדרים בידיים** ואל תדביקו את הערכים האמיתיים שלכם לתוך ה-`chat`. תדביקו את הטקסט בדיוק כמו שהוא. Base44 יזהה אוטומטית את שמות ה-`secrets`, יקפיץ דיאלוג מאובטח, ויבקש מכם להדביק את הערכים שם. כך הסודות נשמרים מוצפנים – לא מופיעים בהיסטוריית הצ'אט שאולי משותפת עם הצוות.

הדביקו ב-Base44 chat:

Install a WhatsApp HTTP connector into this app – DO NOT build any UI yet, just the secrets, helper backend functions, and a Message entity for logging. I'll tell you what to build with it in follow-up prompts.

===== STEP 1 – SECRETS (set these in Base44 → Settings → Secrets BEFORE deploying) =====

```
WA_HUB_URL      = <YOUR_TUNNEL_URL>          # e.g. https://api.example.com
WA_HUB_TOKEN     = <YOUR_HUB_TOKEN>           # the long random string from .env on the server
WA_HUB_SECRET    = <YOUR_WEBHOOK_SECRET>      # the other long random string from .env
```

===== STEP 2 – ENTITY =====

WhatsAppMessage

- direction: enum "incoming" | "outgoing"
- text: text
- ts: number # unix ms timestamp
- external_id: string # message id from the Hub
- chat_number: string # the OTHER side's phone (bare, no +)

RLS: public read/write/create/delete – needed so the anonymous webhook can write.

===== STEP 3 – BACKEND FUNCTIONS (Deno) =====

1. sendWhatsApp(to, text)

- Public function (no auth required).
- Input: { to: string, text: string } – `to` is a phone like "972585802298" (no +, with country code)
- POST to `\${WA\_HUB\_URL}/api/messages/send/text` with
 - headers: { Authorization: `Bearer \${WA\_HUB\_TOKEN}`, "Content-Type": "application/json" }
 - body: JSON.stringify({ to, text })
- On success, log it via asServiceRole.entities.WhatsAppMessage.create({
 - direction: "outgoing", text, ts: Date.now(),
 - external_id: data.id, chat_number: to)
- Return { ok: true, id: data.id } on success, or { ok: false, error } on failure.

2. whatsappWebhook

- Public (the Hub calls this from the internet, no Base44 user auth).
 - Read raw body as TEXT first (do NOT parse JSON yet).
 - Read header "x-hub-signature".
 - Compute expected = "sha256=" + HMAC-SHA256(WA_HUB_SECRET, body).hex()
- IMPORTANT: do NOT use Node's Buffer or timingSafeEqual – they are NOT reliably available in Base44's Deno runtime and will crash with 500.

Use Web Crypto API instead:

```
const enc = new TextEncoder();
const key = await crypto.subtle.importKey(
  "raw", enc.encode(SECRET),
  { name: "HMAC", hash: "SHA-256" }, false, ["sign"]
);
const sig = await crypto.subtle.sign("HMAC", key, enc.encode(body));
const hex = [...new Uint8Array(sig)]
  .map(b => b.toString(16).padStart(2, "0")).join("");
const expected = "sha256=" + hex;
```

```
// simple equal-length string compare is fine for HMAC verification
if (given !== expected) return new Response("bad sig", { status: 401 });
```

- Parse JSON. Handle BOTH directions:


```
IF event === "message.incoming" AND data.type === "text":
  chat_number = data.fromNumber || data.from // the OTHER side
  direction = "incoming"
ELSE IF event === "message.outgoing" AND data.type === "text":
  chat_number = data.chat?.split("@")[0] || data.chat // the OTHER side (recipient)
  direction = "outgoing"
ELSE skip (just return 200).
```
- Then:


```
await asServiceRole.entities.Message.create({
  direction,
  text:      data.text,
  ts:        data.timestamp,
  external_id: data.id,
  chat_number,
});
```
- Return 200 "ok" always (so the Hub doesn't retry on parsing issues that aren't auth).

3. getWhatsAppStatus

- Public.
- GET ``${WA_HUB_URL}/api/instance/status`` with ``Authorization: Bearer ${WA_HUB_TOKEN}``
- Return the JSON as-is.

===== IMPORTANT – gotchas to apply throughout =====

- Base44 ``base44.functions.invoke()`` returns an axios-style wrapper. From the FRONTEND read ``r.data.X`` (not ``r.X``). Easy to miss, will look like "function returned undefined" otherwise.
- Backend entity ops from anonymous functions (the webhook) MUST use ``base44.asServiceRole.entities.X`` – regular ``.entities.X`` requires a logged-in user.
- Use Web Crypto API for HMAC, NOT Node's ``Buffer`` / ``timingSafeEqual`` – Base44's Deno runtime does NOT reliably expose Node's Buffer. Calling it crashes the function with 500. Stick to ``crypto.subtle.sign(...)`` + ``TextEncoder`` as shown in the whatsappWebhook spec above.
- The Message entity MUST have RLS set to public (`{ create:true, read:true, update:true, delete:true }`) so anonymous webhook calls can write to it.
- Phone numbers passed to ``sendWhatsApp`` should be BARE digits with country code: Israeli "0585802298" → "972585802298". No ``+``, no ``0`` at the start.

That's it. Don't build any pages, dashboards, or chat UIs yet. Just confirm the secrets are set, deploy the functions, and tell me the public URL of ``whatsappWebhook`` so I can register it back on my Hub.

■ 8.3 – אחרי שהקונקטור התקין: עכשיו תבנו את האפליקציה שלכם

עכשיו יש לכם 3 פונקציות מוכנות (`getWhatsAppStatus`, `whatsappWebhook`, `sendWhatsApp`) ו-entity-1 לוג. תוכלו לכתוב ל-Base44 כל בקשה שמשתמשת בהן. דוגמאות:

"תבנה לי CRM של לקוחות. בכל קלף לקוח שיהיה כפתור 'שלח WhatsApp' שפותח modal עם textarea, ובלחיצה על שלח מפעיל את הפונקציה `sendWhatsApp` עם המספר של הלקוח."

"תפתח לי דף צ'אט שדומה ל-WhatsApp Web – אני רוצה לראות את כל ה-WhatsAppMessage שיש לי בdb, ממוין לפי ts, מסונן לפי chat_number שאקבל מ-url parameter. למטה תיבת טקסט + כפתור שלח שקורא ל-`sendWhatsApp`."

"כשמגיע incoming WhatsApp מ-whatsappWebhook, אם הטקסט מתחיל ב-'הזמנה', תיצור entity חדש Order מהטקסט ותתייג ב-Slack."

"תוסיף לוח בקרה שמראה: כמה הודעות נשלחו השבוע, כמה התקבלו, ו-status בזמן אמת מ-`getWhatsAppStatus`."

"אתה כבר יודע לדבר עם המספר שלי. אני רוצה שעכשיו יהיה אצלי בוט שמקבל הודעות נכנסות, שולח אותן ל-LLM, וחוזר עם תשובה אוטומטית. רק להודעות שמתחילות עם 'ai!'."

הרעיון: ה-prompt הראשוני מתקין **התשתית**. ה-prompts הבאים בונים את **התוכן**.

■ 8.4 – להגיד ל-Hub שלכם איפה Base44 נמצא

למה צריך את השלב הזה?

יש לכם **שני מערכות** שצריכות לתקשר ביניהן: ה-Base44 ב-cloud וה-Hub שלכם בשרת. הן צריכות לדעת אחת על השנייה כדי שזרימה דו-כיוונית של הודעות תעבוד.

כיוון	מי יוזם	איך מתקנים
Base44 → Hub → WhatsApp יוצא:	כששתמש לוחץ (Base44 שלח)	עובד מעצמו – Base44 כבר יודע את WA_HUB_URL מהסודות
WhatsApp → Hub → Base44 נכנס:	ה-Hub (כשמגיעה הודעה לטלפון)	דורש הגדרה – צריך לתת ל-Hub את ה-URL של ה-webhook ב-Base44

ב-מילים אחרות: **שליחה מהאפליקציה כבר עובדת** אחרי שלב 8.3. בשלב הזה אנחנו מסדרים את הכיוון השני – שהודעות שמישהו ישלח לכם ב-WhatsApp יגיעו ל-Base44.

איך – שלושה צעדים:

צעד 1: שלפו את ה-URL של ה-webhook מ-Base44.

ב-Base44 dashboard → **Functions** → לחיצה על **whatsappWebhook** → חפשו שדה שכתוב בו "Public URL" / "Trigger URL" / "Webhook URL" / "Endpoint URL".

אם לא רואים שדה כזה – אפשר לחלץ ידנית מה-URL בדפדפן

Base44 לפעמים לא מציגים את ה-URL הציבורי בולט בממשק. תוכלו לחלץ אותו מ-URL הדפדפן בכניסה לפרויקט:

```
https://app.base44.com/apps/691f3cf4544082ec29d16b6e/editor/workspace/api...  
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

זה ה-App ID

ה-App ID הוא הקטע בין `/apps/` ל-`/editor/` – מחרוזת hex של 24 תווים.

הפורמט של ה-URL לפונקציה נראה כך:

```
https://app.base44.com/api/apps/<APP_ID>/functions/whatsappWebhook
```

לדוגמה עם ה-App ID למעלה:

```
https://app.base44.com/api/apps/691f3cf4544082ec29d16b6e/functions/whatsappWebhook
```

חלופה: יש פרויקטים ב-Base44 שיש להם sub-domain קבוע (כמו `myapp-1234abcd.base44.app`). אם זה המקרה אצלכם, תוכלו להשתמש ב-`https://.base44.app/api/functions/whatsappWebhook` .ה-`app.base44.com/api/apps/...` למעלה תמיד עובד – לכן מומלץ לנסות אותו קודם.

צעד 2: רישמו את ה-URL הזה ב-Hub (בחלון SSH לשרת – תחליפו את ה-URL במה שהעתקתם):

BASH

```
TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)
WEBHOOK_URL="https://abc123.base44.app/api/functions/whatsappWebhook"

curl -X PUT -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"url":"$WEBHOOK_URL","events":["message.incoming","message.outgoing"]}' \
http://127.0.0.1:3060/api/instance/webhook
```

חשוב: רשמנו שני events – `message.incoming` (הודעות שמגיעות אליך) ו-`message.outgoing` (הודעות שאתה שולח). אם תיקח רק `message.incoming`, הודעות שאתה שולח **מהטלפון** לא יופיעו ב-Base44 (רק הודעות שתשלח מה-Base44 itself, דרך פונקציית `sendWhatsApp`).

תקבלו תגובה כזאת:

JSON

```
{"url":"https://abc123.base44.app/api/functions/whatsappWebhook","events":
["message.incoming","message.outgoing"]}
```

צעד 3: שמרו את ההגדרה גם ב-.env כדי שלא תאבד אם השרת מתחיל מחדש:

BASH

```
sed -i "s|^WEBHOOK_URL=.*|WEBHOOK_URL=$WEBHOOK_URL|" /srv/wa-hub-demo/.env
sed -i "s|^WEBHOOK_EVENTS=.*|WEBHOOK_EVENTS=message.incoming,message.outgoing|" /srv/wa-hub-demo/.env
systemctl restart wa-hub
```

זהו – עכשיו ה-Hub יודע איפה לשלוח כל הודעה שמגיעה ל-WhatsApp שלכם.

■ 8.5 – בדיקה: זרימה דו-כיוונית

ב-Base44, פתחו את הדף שביקשתם מ-Builder לבנות (CRM, צ'אט, מה שיהיה). אז:

1. **שלחו הודעה דרך ה-UI שלכם** → אמורה להגיע ל-WhatsApp תוך 1-2 שניות.

2. **ענו מהטלפון** → תוך 2-3 שניות ה-`whatsappWebhook` יקבל את האירוע, יאמת חתימה, ויכתוב ל-DB. ה-UI שלכם (אם בנוי לזה) יראה את ההודעה.

אם משהו לא עובד – בדקו ב-Base44 dashboard:

• **Functions** → לחיצה על שם הפונקציה → **Logs** – תראו את כל הקריאות וההודעות שגיא.

■ 8.6 – טיפים חשובים (Base44-specific)

בעיה	פתרון
הפרונט מקבל 200 OK אבל מציג undefined	<code>base44.functions.invoke()</code> מחזיר axios wrapper. תקראו <code>r.data.X</code> ולא <code>r.X</code> .
מחזיר 401 למרות webhook חתימה נכונה	בדקו ש- <code>WA_HUB_SECRET</code> ב-Base44 secrets זהה בדיוק ל- <code>WEBHOOK_SECRET</code> ב- <code>.env</code> של השרת. אפילו רווח נוסף ישבור.
מחזיר 401 / רשימה ריקה	ציבורי – RLS דרשתם prompt-הם owner-only ברירת מחדל . ב-Base44 entities <code>asServiceRole</code> אבל וודאו שזה אכן נכנס. אחרת – תקראו דרך פונקציה עם מחזיר <code>Message.list()</code> .
הודעות יוצאות לא נשמרות ב-DB	אותו עניין של RLS. הפונקציה <code>sendMessage</code> חייבת להשתמש ב- <code>asServiceRole.entities.Message.create()</code> .

8.7 – אופציה למפתחים: CLI במקום Builder

אם אתם מעדיפים לערוך קוד מקומית במקום צ'אט עם AI:

BASH

```
npm install -g base44
mkdir wa-chat && cd wa-chat
npx base44 login # פותח דפדפן לאישור
npx base44 create wa-chat -p . -t backend-and-client
```

ואז יוצרים את הקבצים ב- `base44/entities/` , `base44/functions/` , ו- `src/` ידנית – או מבקשים מ- Claude Code/Cursor לעשות זאת. אחרי שינוי:

BASH

```
npx base44 secrets set WA_HUB_URL=... WA_HUB_TOKEN=... WA_HUB_SECRET=... CHAT_NUMBER=...
npm run build
npx base44 deploy -y
```

הקוד המלא של אפליקציה דומה (TypeScript Deno, React) נמצא ב- `examples/base44/` [ברפיו](#).

שלב 8.5 – שליחה בטוחה: מניעת חסימה מ-WhatsApp

WhatsApp לא אוהבים שולחים אוטומטיים. הם משתמשים באלגוריתמים שמזהים התנהגות לא-אנושית ו**חוסמים מספרים**. ספקים מסחריים כמו Green-API משקיעים הרבה בהגנה מפני זה (השהיות, rate-limiting, typing simulation, limit-per-recipient וכו').

ה-Hub הזה לא כולל את ההגנות האלה מהקופסה. אבל זה לא בעיה – אפשר ליישם אותן בקוד שקורא ל-API שלנו. הנה המתכונים החשובים:

■ 8.5.1 – השהייה רנדומלית בין הודעות

הסכנה: שליחת 100 הודעות בלי הפסקה תזוהה בוודאות כספאם. **הפתרון:** המתן 3-15 שניות רנדומליות בין הודעות.

PYTHON

```
import random, time, requests

def send_safe(to, text):
    requests.post(f"{HUB_URL}/api/messages/send/text",
                  headers={"Authorization": f"Bearer {HUB_TOKEN}"},
                  json={"to": to, "text": text})
    # רנדומלי 3-15 שניות – לחיקוי קצב כתיבה אנושי
    time.sleep(random.uniform(3, 15))

for recipient, message in customers:
    send_safe(recipient, message)
```

■ 8.5.2 – סימולציית הקלדה (typing indicator)

לפני שליחת הודעה ארוכה, הראו "...typing" במשך כמה שניות. זה גורם להודעה להיראות אנושית. ה-Hub שלנו לא חושף endpoint לטייפונג ישירות (כי baileys עושה את זה אוטומטית בעבודה רגילה), אבל אפשר להוסיף השהייה בצד שלך:

PYTHON

```
# המתנה כאילו אתה מקליד – 1~100ms
delay = min(len(text) * 0.1, 8) # מקסימום 8 שניות
time.sleep(delay)
send_safe(to, text)
```

■ 8.5.3 Rate limit לפי נמען

לעולם אל תשלחו ליותר מ-1 הודעה ל-30 שניות **לאותו מספר**, ולא יותר מ-20-30 הודעות ביום סך-הכל. שמרו טבלה של "מתי שלחתי לאחרון לכל מספר":

PYTHON

```
last_sent = {} # number -> timestamp

def send_safe_per_user(to, text):
    now = time.time()
    if to in last_sent and now - last_sent[to] < 30:
        wait = 30 - (now - last_sent[to])
        time.sleep(wait)
    send_safe(to, text)
    last_sent[to] = time.time()
```

■ 8.5.4 Warmup למספרים חדשים

מספר שזה עתה נוצר/חובר לא מורגל לתעבורה. שלחו ממנו פחות הודעות בימים הראשונים:

ימים מההתחברות	מקסימום הודעות יוצאות ביום
1-3	10
4-7	30
8-14	100
15+	300+

■ 8.5.5 מתי להפסיק

WhatsApp נותנים אזהרות לפני חסימה – ההודעות שלך **לא מתקבלות עם וי כפול ירוק** אלא נשארות עם וי בודד שעות. אם זה קורה – **תפסיקו לשלוח מיד**, חכו 24 שעות, ושלחו במקצב נמוך משמעותית.

8.5.6 – סיכום: כללי אצבע

למה	כלל
זיהוי ספאם מידי	אל תשלחו ל-100 מספרים תוך 10 דקות
אנושי	השאירו לפחות 30 שניות בין הודעות לאותו מספר
דיווחי ספאם מהמשתמשים – הסיבה #1 לחסימה	אל תשלחו לאנשים שלא ביקשו לקבל הודעות
מגבלה לא רשמית של WhatsApp	לא יותר מ-300 הודעות יוצאות ביום ממספר +1 ימים
סימן אזהרה לפני חסימה	אם וי בודד נשאר שעות – עצור!

רוצים מימוש בתוך ה-Hub? זה ב-roadmap (docs/ROADMAP.md) ברפּו). תרגישו חופשי לפתוח PR או issue ב-GitHub. בינתיים – הקוד שלך הוא המקום הנכון להוסיף את ההגנות (יותר גמיש לתפוזר לפי הצורך).

שלב 9 – אבטחה לפרודקשן

לפני שמשחררים את זה לעולם:

בדיקה	פעולה
HTTPS בלבד?	אם אתם משתמשים ב-Tunnel – כן, אוטומטית
Token חזק?	<code>openssl rand -hex 32</code> נותן 256 ביט – מספיק
?Rate limit	בקובץ <code>.env</code> : <code>RATE_LIMIT_PER_MIN=120</code> . עדכנו לפי הצורך
Webhook ?signed	הקוד מסמן HMAC-SHA256. הצד השני חייב לאמת
השרת מתעדכן?	<code>apt-get install -y unattended-upgrades</code>
ניטור?	<code>/healthz</code> על Uptime Robot + <code>journalctl -u wa-hub -f</code>
גיבוי?	תיקית <code>/srv/wa-hub-demo/data/auth</code> מחזיקה את ה-session. ל-S3/R2 פעם ביום <code>rsync</code>
סודות לא ב-git?	<code>.gitignore</code> כבר חוסם <code>.env</code> . וודאו שלא הוספתם בטעות
תקרת זיכרון?	הוא יחזיר את השירות אוטומטית OOM-כולל <code>MemoryMax=512M</code> ב <code>systemd unit</code>

9.1 – סיבוב Token

אם ה-Token דלף או חשדתם:

BASH

```
NEW=$(openssl rand -hex 32)
sed -i "s/^HUB_TOKEN=.* /HUB_TOKEN=$NEW/" /srv/wa-hub-demo/.env
systemctl restart wa-hub
echo "$NEW"
```

עדכנו ב-Base44 secrets (או היכן שאתם משתמשים) – וזהו. 30 שניות downtime.

■ 9.2 – זיכרון לטווח ארוך

Baileys שומר היסטוריית chats ב-RAM. ב-instance עם מעט תעבורה (פחות מ-100 הודעות/יום) זה לא בעיה. ב-instance פעיל יותר, הזיכרון עלול לטפס.

הגנות שכבר במקום:

- `syncFullHistory: false` ב- `src/baileys/socket.js` – אנחנו לא מורידים את כל ההיסטוריה הישנה
- `markOnlineOnConnect: false` – חוסכים traffic של "אונליין" notifications
- `MemoryMax=512M` ב- `systemd unit` – אם בכל זאת זה תופח מעבר, systemd יחזיר את השירות (Baileys) (ייחבר מחדש)

אם יש לכם volume גבוה במיוחד (אלפי הודעות ביום): שקלו להפחית את ה-cache ב-Baileys, או לפצל ל-`multiple instances`.

טיפים מהשטח – דברים שלא תמצאו בתיעוד

■ #1 Baileys 7 ומספרי LID

החל מ-2025, WhatsApp מציגים **LID (Logical ID)** – מזהה ייחודי לכל משתמש שלא חושף את מספר הטלפון שלו. זה נועד למנוע harvesting של מספרים על-ידי mass-scraping.

```
fromNumber = "972501234567" ← phone: לפני:
fromNumber = "8362502693023" ← LID: אחרי: מספר 13 ספרות מוזר
```

ה-Hub מטפל בזה אוטומטית: קוד `extractPhone()` מנסה לחלץ מספר טלפון אמיתי דרך `senderPn` → `participantPn` → `remoteJidAlt`. אם אין – חוזר ל-LID. שדה `fromLid: true/false` מאותת לכם מהו הזיהוי המקורי.

אם אתם מסננים לפי `fromNumber` ורואים שמסננים יותר מדי – בדקו `fromLid` ב-payload.

■ #2 Base44 functions.invoke מחזיר axios wrapper

JAVASCRIPT

```
const r = await base44.functions.invoke("foo", {});
// r = { data: <actual response>, status: 200, headers: {...} }
```

תקראו `r.data.X` ולא `r.X`. אם הקוד שלכם מתנהג כאילו פונקציה נכשלת אבל ה-network tab מראה OK 200 – זה הסיבה.

■ #3 Base44 entities ברירת מחדל owner-only

ללא בלוק `rls` בסכמה, רק היוצר רואה את הרשומה. webhooks ש-anon נכנסות לא יוכלו לקרוא, גם דרך `asServiceRole` במצב מסוים. הגדירו `rls: { read: true, create: true, ... }` או עברו את כל הקריאות דרך functions עם service role.

■ #4 Webhook URL נמחק ב-restart

`PUT /api/instance/webhook` שומר ב-זיכרון בלבד. שמרו ב-`.env`:

BASH

```
WEBHOOK_URL=https://your-receiver.com/wa
WEBHOOK_EVENTS=message.incoming
```

■ #5 – Quick Tunnel URL מתחדש בכל restart

מצוין לדמו, אסון לפרודקשן. עברו ל-Named Tunnel עם דומיין משלכם ברגע שאתם עוברים מ-experiment ל-production.

■ #6 – 14 Linked Device timeout יום

אם הטלפון הראשי לא היה online במשך 14 יום, WhatsApp מנתקים את כל ה-Linked Devices. תצטרכו פיירינג QR מחדש. אם אתם רוצים שזה יחזיק יותר – ודאו שהטלפון הראשי מחובר לרשת לפחות פעם ב-13 יום.

■ #7 – Windows + Node.js 24 + נתיב עברי = bug ב-base44 0.0.52

על Windows עם שם משתמש עברי (כמו "נעם ניסן"), הגרסה האחרונה של חבילת `base44` ב-npm נכשלת ב-`cp` של `assets`. הפתרון:

BASH

```
npm install --save-dev base44@0.0.51
```

נהוג להתקין global גם:

BASH

```
npm install -g base44@0.0.51
```

(זה ייעלם כשהם יתקנו את ה-bug, אבל נכון לכתיבת המדריך הזה הבעיה קיימת.)

■ #8 – apt-get zombie על שרתי cloud

ראיתי שרת Hetzner שבו `apt-get update` נשאר רץ במשך 11 ימים, חוסם את כל ניסיונות העדכון של `unattended-upgrades`. אם עדכוני אבטחה לא תופסים – בדקו:

BASH

```
ps -ef | grep apt-get | grep -v grep
# אם רואים תהליך מ-3+ ימים – הרגו:
kill -9 <PID>
rm -f /var/lib/dpkg/lock-frontend /var/lib/dpkg/lock /var/cache/apt/archives/lock
apt-get update
```


פתרון בעיות נפוצות

השירות מופעל אבל QR לא מופיע ▼

BASH

```
journalctl -u wa-hub -n 50 --no-pager
```

חפשו `QR generated` . אם לא מופיע – בדקו שיש חיבור לאינטרנט:

BASH

```
curl -sS https://web.whatsapp.com | head -1
```

"already paired" אבל בטלפון לא רואים את ההתקן ▼

ה-Hub חושב שהוא מחובר אבל בעצם לא. אפסו:

BASH

```
curl -X POST -H "Authorization: Bearer $TOKEN" \
http://127.0.0.1:3060/api/instance/logout
```

ובקשו QR חדש.

הודעות לא נשלחות – not_connected ▼

ב-95% מהמקרים זה כי הטלפון לא היה מחובר לאינטרנט יותר מ-14 יום, וה-session פג. הריצו את תהליך הפיירינג מחדש מהשלב 5.

Cloudflare Tunnel נופל ▾

BASH

```
systemctl status cloudflared-wahub # Quick tunnel
systemctl status cloudflared # Named tunnel
journalctl -u cloudflared-wahub -n 50 --no-pager
```

לרוב זה שינוי ב-URL (ב-Quick Tunnel) – restart ייצור URL חדש. שמרו את ה-URL החדש ועדכנו בכל מקום שהשתמשתם בו.

השירות נופל ב-SYS=31/status • core-dump ▾

זה systemd seccomp filter – SIGSYS חוסם syscall ש-Node 20+ צריך (`io_uring` , `clone3`). לוג יראה משהו כמו:

```
wa-hub.service: Main process exited, code=dumped, status=31/SYS
wa-hub.service: Failed with result 'core-dump'.
```

תיקון: הסירו את ה-SystemCallFilter דרך drop-in:

BASH

```
mkdir -p /etc/systemd/system/wa-hub.service.d
printf '[Service]\nSystemCallFilter=\n' > /etc/systemd/system/wa-hub.service.d/syscall-
fix.conf
systemctl daemon-reload && systemctl restart wa-hub.service
```

שאר ההגנות (`NoNewPrivileges` , `ProtectSystem=strict`) וכו' נשארות בתוקף.

Webhook לא נכנס ל-Base44 (OK 200 בלוג של Hub, אבל ה-entity ריק) ▾

1. ראו את הלוגים של הפונקציה: `npx base44 logs --function whatsappWebhook --limit 20`

2. אם רואים שגיאת RLS – וודאו שב- `whatsappmessage.jsonc` יש

```
rls: { create: true, read: true, ... }
```

3. אם רואים שגיאת חתימה – בדקו ש- `WA_HUB_SECRET` ב-Base44 secrets זהה לחלוטין ל-

`WEBHOOK_SECRET` ב- `.env` של השרת

הודעות נכנסות יש, אבל fromNumber הוא LID ▼

ראו "טיפ #1" למעלה. השדה `fromLid: true` מאותת שזה LID. אם רוצים את מספר הטלפון האמיתי – אין דרך, WhatsApp לא חושפים את זה. תזהו לקוחות לפי `chat` (ה-JID של השיחה) שיציב לאורך זמן.

הצעדים הבאים

- **רוצים multi-tenant?** הוסיפו טבלת tenants ל-DB → לכל לקוח `instance_id` ו-port נפרד. אפשרות נוספת: להריץ `wa-hub-demo` כמה פעמים בקונטיינרים על אותו שרת.
- **רוצים AI אוטומטי?** הוסיפו פונקציה שמקבלת `message.incoming`, שולחת ל-Claude/GPT, ומחזירה תגובה חזרה דרך `POST /api/messages/send/text`. צ'אטבוט תוך 30 שורות.
- **רוצים התראות לצוות?** ב- `/api/instance/webhook` אפשר להגדיר Slack/Discord webhook במקום (או בנוסף ל-) Base44.
- **רוצים dashboard?** ה-WebSocket ב- `:3061` משדר כל אירוע בזמן אמת – חברו לאפליקציית React/Vue/Svelte.
- **רוצים סיכומים יומיים?** `cron` + `curl` ב- `/etc/cron.daily/`.

כלים שנדאי להכיר

- [Baileys](#) – הספרייה שעושה את הקסם של WhatsApp Web
 - [Cloudflared](#) – Tunnel
 - [Express](#) – REST framework
 - [Hetzner Cloud](#) – VPS מומלץ
 - [Uptime Robot](#) – חינום לניטור 50 endpoints
 - [Cloudflare R2](#) – אחסון זול לגיבוי `data/auth`
-

שאלות נפוצות

- Q: זה חוקי? A:** כן. WhatsApp לא אוסר על Linked Devices לא-רשמיים, אבל שומר לעצמו את הזכות לחסום מספרים שמתנהגים כספאם. שלחו רק להודעות שביקשו מכם.
- Q: השרת יעמוד בעומס? A:** CX23 / CAX11 (4GB RAM) טובים עד ~50 הודעות לשנייה. אם אתם מצפים ליותר – שדרגו ל-CX33 / CAX21 (8GB), או הריצו כמה instances.
- Q: מה קורה אם Hetzner נופל? A:** ב-12 חודשים אחרונים – uptime ~99.95%. לפרודקשן רצינית שקלו multi-region או fallback ל-cloud אחר.
- Q: מה הסיכון שיחסמו לי את המספר? A:** נמוך אם אתם משתמשים בו כמו אדם רגיל – תגובות, סיכומים, לא יותר מ-1000 הודעות ביום למספרים אחרים. גבוה אם אתם שולחים cold outreach.
- Q: יש Web Crypto במקום node:crypto? A:** כן – ראו דוגמה ב-8.2 (ה-prompt של whatsappWebhook) לסביבות שלא תומכות ב-Node modules (Cloudflare Workers, Vercel Edge, Base44 Deno וכו'). שני הפתרונות עובדים זהה לחתימת HMAC.
- Q: אפשר להשתמש בקוד מסחרי? A:** כן. רישיון MIT – תפרק, תשנה, תפרסם, תמכור. אם תרגישו בנוח, תזכירו את הפרויקט המקורי.

נספח – נהלי חירום לוובר

חמש התקלות הכי נפוצות בדמו חי – והפקודה האחת שמתקנת כל אחת. שמרו את העמוד הזה פתוח על מסך נפרד בזמן ההרצה.

תקלה	סימן בשטח	תיקון מיידי (שורה אחת על השרת)
ה-Hub לא רץ	<code>curl 127.0.0.1:3060/healthz</code> מחזיר <code>connection refused</code>	<code>nalctl -u wa-hub -n 20 --no-pager</code>
URL של Tunnel השתנה	בקשות חיצוניות מקבלות 530/1033 פתאום	<code>]+\trycloudflare\.com' head -1</code>
QR פג תוקף לפני שסרקתם	בטלפון "קוד לא תקין"	חזרו על 2 הפקודות ב-5.2§ Enter) Enter →
	הלוג צועק <code>loggedOut</code> , אין QR חדש	<code>27.0.0.1:3060/api/instance/logout</code>
השירות נופל ב-	systemd: <code>code=dumped, status=31/SYS</code>	<code>eload && systemctl restart wa-hub</code> <code>status=31/SYS</code>

■ תדריך 60-שניות לפני שמתחילים

- SSH פתוח לשרת בטרמינל אחד, PowerShell מקומי בטרמינל שני.
- `TOKEN=$(grep HUB_TOKEN /srv/wa-hub-demo/.env | cut -d= -f2)` כבר רץ ב-SSH.
- `curl -sS http://127.0.0.1:3060/healthz` מחזיר `{"ok":true}`.
- ה-Tunnel URL מועתק לקליפבורד)
`journalctl -u cloudflared-wahub | grep trycloudflare | head -1` .
- הטלפון פתוח ב-WhatsApp → הגדרות → מכשירים מקושרים, מוכן לסרוק.
אם 1-5 ✓ – אפשר להתחיל. אם משהו אדום – שורה רלוונטית בטבלה למעלה תסדר את זה בפחות מ-30 שניות.

יש לכם עכשיו תשתית WhatsApp עצמאית מקצועית.

קוד פתוח: github.com/Noam13-w/wa-hub-demo